

PYTHON VORKURS

Übungsheft für 10 Kurstage

Anfängerfreundlich · auf Deutsch · mit 60 Aufgaben

Für wen?

Für Teilnehmende ohne oder mit wenig Programmiererfahrung. Pro Kurstag sind etwa 90–120 Minuten vorgesehen.

Arbeitsweise

Erst vorhersagen und planen, dann selbst schreiben, testen und verbessern. Musterlösungen sind bewusst nicht enthalten: Der Lösungsweg zählt.

Name: _____

Kurs / Gruppe: _____

So arbeitest du mit diesem Heft

Jeder Tag beginnt mit einem kleinen Syntaxfenster. Danach folgen drei leichte Aufgaben, zwei normale Aufgaben und eine Herausforderung. Du musst nicht jede Herausforderung lösen. Wichtig ist, dass du die Kernaufgaben selbst ausprobierst.

Drei Schritte

1. Verstehen: Markiere Eingabe, Verarbeitung und Ausgabe. 2. Planen: Schreibe Stichpunkte oder Pseudocode. 3. Testen: Probiere normale Werte, Grenzwerte und Fehleingaben.

Wenn etwas nicht funktioniert

Lies die letzte Zeile der Fehlermeldung. Prüfe danach Klammern, Anführungszeichen, Doppelpunkte, Einrückung und Datentypen. Ändere nur eine Sache auf einmal und starte erneut.

Kursübersicht

Tag	Thema	Kernidee
1	Erste Programme: Ausgabe und Syntax	Anweisungen von oben nach unten lesen, Text ausgeben, Kommentare verwenden.
2	Variablen und Datentypen	Werte speichern, verändern und als int, float, str oder bool einordnen.
3	Eingabe und Operatoren	input() verwenden, Texte in Zahlen umwandeln und Rechenoperatoren sicher einsetzen.
4	Bedingungen: if, elif, else	Vergleiche formulieren und genau einen passenden Programmzweig ausführen.
5	Schleifen: for und while	Wiederholungen mit Zähler, range() und Abbruchbedingung steuern.
6	Listen	Mehrere Werte speichern, per Index lesen, ergänzen und durchlaufen.
7	Zeichenketten	Strings indizieren, zerlegen, normalisieren und Zeichen zählen.
8	Funktionen	Aufgaben in benannte Bausteine zerlegen, Parameter nutzen und Werte zurückgeben.
9	Konzepte verbinden und Eingaben prüfen	Listen, Schleifen, Bedingungen und Funktionen zu robusten Programmen kombinieren.
10	Mini-Projekt	Ein kleines Programm planen, schrittweise bauen, testen und kurz vorstellen.

Tag 1 · Erste Programme: Ausgabe und Syntax

Lernziel

Anweisungen von oben nach unten lesen, Text ausgeben, Kommentare verwenden.

Syntaxfenster

```
print("Hallo, Welt!")  
  
# Ein Kommentar  
print("Python startet!")
```

Aufgaben

LEICHT

1. Drei Zeilen

Gib deinen Namen, dein Studienfach und einen freundlichen Gruß in drei getrennten Zeilen aus.

Beispiel/Test: Drei sichtbare Ausgabezeilen.

LEICHT

2. Ausgabe vorhersagen

Notiere zuerst auf Papier, was drei aufeinanderfolgende print()-Anweisungen ausgeben. Führe den Code danach aus.

Beispiel/Test: Stimmt deine Vorhersage exakt?

LEICHT

3. Mini-Steckbrief

Erzeuge einen Steckbrief mit Überschrift und vier Angaben. Nutze zunächst nur print().

Beispiel/Test: Keine Eingabe erforderlich.

NORMAL

4. Kassenbon

Gib einen kleinen Kassenbon mit drei Artikeln und einer Gesamtsumme aus. Richte ihn lesbar aus.

Beispiel/Test: Die Ausgabe soll wie ein einfacher Beleg wirken.

NORMAL

5. Fehlerdetektiv

Korrigiere fünf vorbereitete Fehler: fehlendes Anführungszeichen, falsche Klammer, falsche Großschreibung, zusätzlicher Buchstabe und fehlende Klammer.

Beispiel/Test: Ändere jeweils nur das Nötigste.

HERAUSFORDERUNG

6. ASCII-Bild

Zeichne mit mehreren print()-Zeilen ein Haus, Tier oder Gesicht aus Tastaturzeichen.

Beispiel/Test: Mindestens 6 Zeilen und ein Kommentar.

Tag 2 · Variablen und Datentypen

Lernziel

Werte speichern, verändern und als int, float, str oder bool einordnen.

Syntaxfenster

```
name = "Mina"
alter = 19
groesse = 1.72
ist_studentin = True

breite = 5
hoehe = 3
flaeche = breite * hoehe
print(flaeche)
```

Aufgaben

LEICHT

1. Vier Typen

Lege je eine Variable für Name, Alter, Körpergröße und Studierendenstatus an. Gib Wert und `type(...)` aus.

Beispiel/Test: Erwartet werden str, int, float und bool.

LEICHT

2. Rechteck

Speichere Breite und Höhe. Berechne Fläche und Umfang.

Beispiel/Test: Für 5 und 3: Fläche 15, Umfang 16.

LEICHT

3. Minuten

Wandle 7 Minuten in Sekunden um. Nutze Variablen.

Beispiel/Test: Ergebnis: 420 Sekunden.

NORMAL

4. Temperatur

Wandle 20 Grad Celsius mit $F = C * 9 / 5 + 32$ in Fahrenheit um.

Beispiel/Test: Ergebnis: 68.0.

NORMAL

5. Satz mit Werten

Baue mit einem f-String einen Satz aus mindestens drei Variablen.

Beispiel/Test: Keine Plus-Zeichen zwischen Text und Zahlen nötig.

HERAUSFORDERUNG

6. Zeit zerlegen

Zerlege 3672 Sekunden mit `//` und `%` in Stunden, Minuten und Restsekunden.

Beispiel/Test: 1 h, 1 min, 12 s.

Tag 3 · Eingabe und Operatoren

Lernziel

input() verwenden, Texte in Zahlen umwandeln und Rechenoperatoren sicher einsetzen.

Syntaxfenster

```
name = input("Name: ")
print(f"Hallo, {name}!")

zahl = int(input("Zahl: "))
print(zahl * 2)
```

Aufgaben

LEICHT

1. Begrüßung

Frage Name und Wohnort ab und gib eine persönliche Begrüßung aus.

Beispiel/Test: Eingabe: Ada, Berlin.

LEICHT

2. Zwei Zahlen

Lies zwei ganze Zahlen ein und gib Summe, Differenz und Produkt aus.

Beispiel/Test: Teste mit 10 und 4.

LEICHT

3. Rechteck interaktiv

Lies Breite und Höhe als float ein und berechne die Fläche.

Beispiel/Test: Teste auch 2.5 und 4.

NORMAL

4. Celsius-Rechner

Lies Celsius ein und gib Fahrenheit aus. Runde zunächst nicht.

Beispiel/Test: 0 °C → 32 °F.

NORMAL

5. Sekunden-Rechner

Lies eine Sekundenzahl ein und zerlege sie wie an Tag 2.

Beispiel/Test: 3672 → 1:01:12.

HERAUSFORDERUNG

6. Mini-Rechner

Lies zwei Zahlen ein und zeige +, -, *, / sowie Rest %. Weise bei der Division im Ausgabebetext auf den Divisor hin.

Beispiel/Test: Teste 10 und 4; Division 2.5.

Tag 4 · Bedingungen: if, elif, else

Lernziel

Vergleiche formulieren und genau einen passenden Programmzweig ausführen.

Syntaxfenster

```
zahl = int(input("Zahl: "))
if zahl > 0:
    print("positiv")
elif zahl < 0:
    print("negativ")
else:
    print("null")

if alter >= 18 and hat_ticket:
    print("Einlass")
```

Aufgaben

LEICHT

1. Positiv, negativ oder null

Ordne eine eingegebene Zahl einer der drei Kategorien zu.

Beispiel/Test: Teste -2, 0 und 8.

LEICHT

2. Gerade oder ungerade

Nutze den Restoperator, um eine ganze Zahl zu prüfen.

Beispiel/Test: 8 gerade; 7 ungerade.

LEICHT

3. Größere Zahl

Lies zwei Zahlen ein und gib die größere aus. Behandle Gleichheit.

Beispiel/Test: 5 und 5 → gleich groß.

NORMAL

4. Notenstufen

Ordne Punkte 90–100, 80–89, 70–79, 60–69 und darunter Kategorien zu. Prüfe zuerst ungültige Werte.

Beispiel/Test: 101 und -1 sind ungültig.

NORMAL

5. Ticketpreis

Bestimme anhand des Alters: unter 6 kostenlos, bis 17 ermäßigt, bis 64 regulär, sonst Seniorentarif.

Beispiel/Test: Prüfe Grenzwerte 5/6/17/18/64/65.

HERAUSFORDERUNG

6. Schaltjahr

Ein Jahr ist Schaltjahr, wenn es durch 4 teilbar ist, außer durch 100 - es sei denn, auch durch 400.

Beispiel/Test: 1900 nein, 2000 ja, 2024 ja.

Tag 5 · Schleifen: for und while

Lernziel

Wiederholungen mit Zähler, range() und Abbruchbedingung steuern.

Syntaxfenster

```
for zahl in range(1, 6):  
    print(zahl)  
  
rest = 3  
while rest > 0:  
    print(rest)  
    rest -= 1
```

Aufgaben

LEICHT

1. Zählen

Gib mit for die Zahlen 1 bis 10 aus.

Beispiel/Test: Achte auf die exklusive Obergrenze von range().

LEICHT

2. Gerade Zahlen

Gib alle geraden Zahlen von 2 bis 20 aus.

Beispiel/Test: Nutze range() mit Schrittweite.

LEICHT

3. Countdown

Zähle mit while von 10 bis 1 und gib danach Los! aus.

Beispiel/Test: Die Schleife muss enden.

NORMAL

4. Einmaleins

Lies n ein und gib n * 1 bis n * 10 aus.

Beispiel/Test: Für n=3 endet die Tabelle bei 30.

NORMAL

5. Summe 1 bis n

Berechne die Summe mit Schleife, nicht mit einer Formel.

Beispiel/Test: n=5 → 15.

HERAUSFORDERUNG

6. Passwortversuche

Erlaube höchstens drei Eingaben. Bei richtigem Passwort endet die Schleife sofort, sonst folgt eine Sperrmeldung.

Beispiel/Test: Teste Erfolg beim 1. und 3. Versuch sowie dreimal falsch.

Tag 6 · Listen

Lernziel

Mehrere Werte speichern, per Index lesen, ergänzen und durchlaufen.

Syntaxfenster

```
zahlen = [4, 8, 2, 9]
print(zahlen[0])
zahlen.append(5)

for zahl in zahlen:
    print(zahl)
```

Aufgaben

LEICHT

1. Elemente ausgeben

Lege eine Liste mit fünf Lieblingsessen an und gib jedes Element einzeln aus.

Beispiel/Test: Nutze eine Schleife.

LEICHT

2. Liste verändern

Füge ein Element mit `append()` hinzu und entferne eines mit `remove()`.

Beispiel/Test: Gib vorher und nachher aus.

LEICHT

3. Summe bilden

Addiere alle Zahlen einer Liste mit eigener Schleife.

Beispiel/Test: Vergleiche erst danach mit `sum()`.

NORMAL

4. Durchschnitt

Berechne `Summe / len(liste)`. Behandle eine leere Liste mit einer Bedingung.

Beispiel/Test: `[2,4,6] → 4.0`.

NORMAL

5. Maximum selbst finden

Starte mit dem ersten Listenelement und vergleiche alle weiteren. `max()` ist hier tabu.

Beispiel/Test: `[-5,-2,-9] → -2`.

HERAUSFORDERUNG

6. Notenstatistik

Ermittle aus fünf Punktzahlen Durchschnitt, Maximum, Minimum und Anzahl bestandener Werte ab 50.

Beispiel/Test: Nutze mindestens eine Schleife.

Tag 7 · Zeichenketten

Lernziel

Strings indizieren, zerlegen, normalisieren und Zeichen zählen.

Syntaxfenster

```
text = "Python"
print(text[0])
print(text[-1])
print(text[1:4])

wort = input("Wort: ").strip().lower()
```

Aufgaben

LEICHT

1. Zeichen einzeln

Gib jedes Zeichen eines eingegebenen Wortes in einer eigenen Zeile aus.

Beispiel/Test: Python ergibt 6 Zeilen.

LEICHT

2. Länge und Rand

Gib Länge, erstes und letztes Zeichen aus. Prüfe vorher, ob der Text leer ist.

Beispiel/Test: Leere Eingabe darf nicht abstürzen.

LEICHT

3. Groß und klein

Zeige denselben Text mit upper(), lower() und title().

Beispiel/Test: Beobachte Leerzeichen.

NORMAL

4. Vokale zählen

Zähle a, e, i, o, u unabhängig von Großschreibung.

Beispiel/Test: Informatik → 4.

NORMAL

5. Text umdrehen

Gib einen Text rückwärts aus - zuerst mit Slicing, freiwillig zusätzlich mit Schleife.

Beispiel/Test: Python → nohtyP.

HERAUSFORDERUNG

6. Palindrom

Ignoriere Groß-/Kleinschreibung und äußere Leerzeichen. Vergleiche normalisierten Text mit seiner Umkehrung.

Beispiel/Test: Anna → Palindrom; Python → keines.

Tag 8 · Funktionen

Lernziel

Aufgaben in benannte Bausteine zerlegen, Parameter nutzen und Werte zurückgeben.

Syntaxfenster

```
def begruesse(name):  
    print(f"Hallo, {name}!")  
  
def addiere(a, b):  
    return a + b
```

Aufgaben

LEICHT

1. Quadrat

Schreibe `quadrat(zahl)`, das das Quadrat zurückgibt.

Beispiel/Test: `quadrat(4)` → 16.

LEICHT

2. Begrüßung

Schreibe `begruesse(name)`, das einen Begrüßungstext zurückgibt.

Beispiel/Test: Rückgabe statt Eingabe in der Funktion.

LEICHT

3. Gerade prüfen

Schreibe `ist_gerade(zahl)` mit bool-Rückgabe.

Beispiel/Test: `ist_gerade(7)` → False.

NORMAL

4. Maximum

Schreibe `maximum(a, b)`, ohne `max()` zu verwenden. Gleichheit soll erlaubt sein.

Beispiel/Test: `maximum(-2, -5)` → -2.

NORMAL

5. Temperaturfunktion

Schreibe `celsius_zu_fahrenheit(c)`. Teste mindestens drei Werte.

Beispiel/Test: 0 → 32; 100 → 212.

HERAUSFORDERUNG

6. Rechner zerlegen

Erstelle `addieren`, `subtrahieren`, `multiplizieren` und `dividieren`. Bei Division durch 0 soll eine verständliche Rückgabe entstehen.

Beispiel/Test: Jede Funktion separat testen.

Tag 9 · Konzepte verbinden und Eingaben prüfen

Lernziel

Listen, Schleifen, Bedingungen und Funktionen zu robusten Programmen kombinieren.

Syntaxfenster

```
try:
    alter = int(input("Alter: "))
except ValueError:
    print("Bitte eine ganze Zahl eingeben.")

while True:
    befehl = input("> ").strip().lower()
    if befehl == "ende":
        break
```

Aufgaben

LEICHT

1. Sichere Ganzzahl

Wiederhole die Eingabe, bis `int(...)` gelingt. Gib danach die Zahl aus.

Beispiel/Test: Teste Text, Dezimalzahl und ganze Zahl.

LEICHT

2. Wortliste

Lies Wörter ein, bis `ende` eingegeben wird. Gib danach die Liste aus.

Beispiel/Test: `ende` gehört nicht zur Liste.

LEICHT

3. Menü

Zeige ein Menü wiederholt und beende nur bei Auswahl 0. Unbekannte Auswahl erhält eine Meldung.

Beispiel/Test: Teste 1, 9 und 0.

NORMAL

4. Einkaufsliste

Menü: hinzufügen, anzeigen, entfernen, beenden. Verhindere Fehler beim Entfernen unbekannter Einträge.

Beispiel/Test: Liste bleibt während der Schleife erhalten.

NORMAL

5. Zahlen raten

Nutze zunächst eine feste Geheimzahl. Gib zu klein, zu groß oder richtig aus und zähle Versuche.

Beispiel/Test: Geheimzahl 7; mehrere Testwege.

HERAUSFORDERUNG

6. Quiz-Runde

Speichere Fragen und Antworten in parallelen Listen oder als Liste von Paaren. Zähle Punkte und gib am Ende eine Quote aus.

Beispiel/Test: Mindestens 5 Fragen; Großschreibung ignorieren.

Tag 10 · Mini-Projekt

Lernziel

Ein kleines Programm planen, schrittweise bauen, testen und kurz vorstellen.

Syntaxfenster

```
# 1. Ziel in einem Satz
# 2. Eingaben und Ausgaben
# 3. Teilaufgaben als Funktionen
# 4. Testfälle

def main():
    ...

main()
```

Aufgaben

LEICHT

1. Projekt wählen

Wähle Quiz, To-do-Liste, Rechner, Zahlenraten oder Schere-Stein-Papier. Formuliere Ziel und Mindestfunktionen.

Beispiel/Test: Das Projekt ist in 60–90 Minuten machbar.

LEICHT

2. Ablauf skizzieren

Schreibe Pseudocode: Start, Eingaben, Verarbeitung, Ausgabe, Wiederholung, Ende.

Beispiel/Test: Noch kein Python-Code nötig.

LEICHT

3. Testplan

Notiere mindestens fünf Testfälle, darunter Grenz- oder Fehleingaben.

Beispiel/Test: Eingabe und erwartete Ausgabe festhalten.

NORMAL

4. Mindestversion bauen

Implementiere eine lauffähige Version mit mindestens einer Schleife, einer Bedingung, einer Liste und zwei eigenen Funktionen.

Beispiel/Test: Kleine funktionierende Version zuerst.

NORMAL

5. Benutzbarkeit verbessern

Ergänze klare Prompts, verständliche Fehlermeldungen und eine saubere Abschlussausgabe.

Beispiel/Test: Eine andere Person kann es ohne Erklärung bedienen.

HERAUSFORDERUNG

6. Erweiterung

Ergänze genau eine sinnvolle Funktion: Zufall, Highscore, mehrere Runden, Statistik oder Schwierigkeitsgrad.

Beispiel/Test: Die Mindestversion muss weiterhin funktionieren.

Abschlussprojekt · Checkliste

Nutze diese Checkliste am Tag 10. Ein kleines vollständiges Projekt ist besser als ein großes unfertiges.

Mindestumfang

Eine wiederholbare Hauptschleife · mindestens eine if/elif/else-Entscheidung ·
mindestens eine Liste · mindestens zwei eigene Funktionen · verständliche
Ein- und Ausgaben.

Vor der Abgabe

- ☐ Programm startet ohne Fehler.
- ☐ Alle Menüwege wurden mindestens einmal getestet.
- ☐ Ungültige Eingaben führen nicht sofort zum Absturz.
- ☐ Variablen und Funktionen haben verständliche Namen.
- ☐ Doppelte Logik wurde möglichst in Funktionen ausgelagert.
- ☐ Die Ausgabe erklärt, was als Nächstes möglich ist.
- ☐ Eine andere Person hat das Programm ausprobiert.

Kurzvorstellung (2 Minuten)

1. Was macht das Programm? 2. Welche Eingabe wird erwartet? 3. Zeige einen normalen Ablauf. 4. Nenne eine Schwierigkeit und deine Lösung.

Python-Cheatsheet

Die wichtigsten Bausteine dieses Vorkurses auf wenigen Seiten.

Ausgabe, Kommentare und Variablen

```
print("Hallo")
# Kommentar
name = "Mira"
alter = 20
print(f"{name} ist {alter} Jahre alt.")
```

Datentypen und Umwandlung

```
ganze_zahl = 12 # int
kommazahl = 3.5 # float
text = "Python" # str
wahrheitswert = True # bool

int("12") float("3.5") str(12) type(text)
```

Eingabe

```
name = input("Name: ") # liefert str
alter = int(input("Alter: ")) # Umwandlung zu int
```

Rechnen und Vergleichen

Operator	Bedeutung	Beispiel
+ - * /	Grundrechenarten	$7 / 2 \rightarrow 3.5$
//	Ganzzahldivision	$7 // 2 \rightarrow 3$
%	Rest	$7 \% 2 \rightarrow 1$
**	Potenz	$2 ** 3 \rightarrow 8$
== !=	gleich / ungleich	$x == 5$
< <= > >=	Vergleiche	$alter \geq 18$
and or not	Logik	$x > 0 \text{ and } x < 10$

Python-Cheatsheet · Steuerung

Bedingungen

```
if punkte >= 90:
    print("sehr gut")
elif punkte >= 50:
    print("bestanden")
else:
    print("noch nicht bestanden")
```

for-Schleife und range()

```
for i in range(5): # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 11, 2): # 2, 4, 6, 8, 10
    print(i)
```

while-Schleife

```
versuche = 3
while versuche > 0:
    print(versuche)
    versuche -= 1
```

Einrückung

Nach if, elif, else, for, while und def steht ein Doppelpunkt. Der zugehörige Block wird gleichmäßig eingerückt - üblich sind vier Leerzeichen.

Listen

```
namen = ["Ada", "Linus"]
namen.append("Grace")
print(namen[0])
print(len(namen))

for name in namen:
    print(name)
```

Nützliche Listenoperationen

```
liste.append(x) # hinzufügen
liste.remove(x) # ersten passenden Wert entfernen
x in liste # Mitgliedschaft prüfen
len(liste) # Anzahl
sum(zahlen) # Summe
min(zahlen), max(zahlen)
```

Python-Cheatsheet · Strings und Funktionen

Strings

```
text = input("Text: ").strip()
print(text.lower())
print(text.upper())
print(text[0]) # erstes Zeichen
print(text[-1]) # letztes Zeichen
print(text[::-1]) # rückwärts
print(len(text))
```

Funktionen

```
def flaeche(breite, hoehe):
    ergebnis = breite * hoehe
    return ergebnis

print(flaeche(5, 3))
```

Fehler bei Eingaben abfangen

```
try:
    zahl = int(input("Ganze Zahl: "))
except ValueError:
    print("Das war keine ganze Zahl.")
```

Häufige Fehlermeldungen

Meldung	Oft bedeutet das
SyntaxError	Schreibfehler, fehlender Doppelpunkt, Klammer oder Anführungszeichen
IndentationError	Einrückung fehlt oder ist uneinheitlich
NameError	Variablen- oder Funktionsname ist unbekannt / anders geschrieben
TypeError	Operation passt nicht zu den Datentypen
ValueError	Umwandlung ist grundsätzlich möglich, aber dieser Wert passt nicht
IndexError	Listen- oder String-Index liegt außerhalb des gültigen Bereichs

Deutsch im Programmierunterricht

Du musst englische Codewörter nicht übersetzen: Gesprochen wird deutsch, Python bleibt Python. Hilfreich sind feste Formulierungen, die du wiederholt verwendest.

Begriffe

Englisch / Code	Gute deutsche Formulierung
variable	Variable; speichert einen Wert
assignment	Zuweisung
statement	Anweisung
expression	Ausdruck
condition	Bedingung
branch	Programmzweig / Fall
loop / iteration	Schleife / Schleifendurchlauf
list / index	Liste / Index
string	Zeichenkette; im Gespräch oft auch String
function / parameter	Funktion / Parameter
argument	Argument bzw. eingesetzter Wert
return value	Rückgabewert
bug / debugging	Fehler / Fehlersuche bzw. Debugging
exception	Ausnahme / Laufzeitfehler
indentation	Einrückung

Nützliche Tutor-Sätze

- Was soll das Programm als Eingabe bekommen?
- Was soll am Ende ausgegeben werden?
- Welche Teilaufgaben erkennst du?
- Sage zuerst in eigenen Worten, was die Schleife wiederholt.
- Welche Werte sollten wir zum Testen wählen?
- Lies die letzte Zeile der Fehlermeldung laut vor.
- Was hast du erwartet - und was ist tatsächlich passiert?
- Zeige mir die kleinste Stelle, an der das Ergebnis falsch wird.

Weiterlernen: eine sinnvolle Auswahl

CS50P noch einmal komplett ansehen? Wahrscheinlich nicht. Für die Vorbereitung dieses zweiwöchigen Vorkurses reicht ein gezielter Auffrischungsdurchgang: Functions & Variables, Conditionals, Loops und Exceptions. Sieh die Vorlesungen abschnittsweise und löse jeweils ein bis zwei kleine Aufgaben selbst. Libraries, Unit Tests, File I/O, Regular Expressions und OOP gehen deutlich über den hier geplanten Kern hinaus.

Empfehlung

Plane 4 kurze Wiederholungsblöcke à 45–75 Minuten. Schreibe bei jedem Block ein Mini-Programm ohne Vorlage. Für Tutorinnen und Tutoren ist aktives Erklären wichtiger als passives Wiederansehen.

Deutschsprachige Ressourcen

openHPI: „Python - schnell und intensiv Programmieren lernen“

Deutschsprachiger Selbstlernkurs mit Übungen und automatischem Feedback. Nutze besonders Abschnitte zu Datentypen, Funktionen, Bedingungen, Schleifen und Fehlerbehandlung.

<https://open.hpi.de/learn/python2024?locale=de>

python-kurs.eu

Frei zugängliches deutsches Nachschlagewerk mit vielen Beispielen und Übungen. Gut, um Formulierungen und Fachwortschatz für einzelne Unterrichtsthemen vorzubereiten.

<https://www.python-kurs.eu/>

CS50P

Didaktisch stark und sehr übungsorientiert, aber englisch. Nutze es gezielt für deine fachliche Auffrischung; das deutsche Vokabular trainierst du parallel mit den beiden Quellen oben.

<https://cs50.harvard.edu/python/>

So trainierst du die deutsche Erklärung

Wähle pro Kurstag fünf Begriffe aus der Begriffstabelle. Erkläre den Tagescode zwei Minuten laut auf Deutsch, ohne ihn nur vorzulesen. Nimm dich einmal auf und prüfe: Sind Eingabe, Verarbeitung und Ausgabe klar? Verwendest du dieselben Begriffe konsistent?

Stand der Links: August 2026.